

Model Enumeration using IPASIR-UP without Blocking Clauses

Timpe Hörig

June 15, 2026

Model Enumeration using IPASIR-UP without Blocking Clauses

Model Enumeration

$$\text{cnf: } (X \vee Y) \wedge (\bar{Y} \vee Z)$$

$$\text{solution 1: } X \wedge Y \wedge Z$$

$$\text{solution 2: } \bar{X} \wedge Y \wedge Z$$

$$\text{solution 3: } X \wedge \bar{Y} \wedge Z$$

$$\text{solution 4: } X \wedge \bar{Y} \wedge \bar{Z}$$

Model Enumeration using Blocking Clauses

Pseudo Code

```
list[model] ms
while solver.solve(cnf) == SAT:
    model m = solver.solution()
    ms += [m]
    cnf += not(m)
```

Model Enumeration using Blocking Clauses

Blocking Clause

$$cnf : (X \vee Y) \wedge (\bar{Y} \vee Z)$$

$$model : X \wedge Y \wedge Z$$

$$\text{not}(model) : \neg(X \wedge Y \wedge Z) \iff \bar{X} \vee \bar{Y} \vee \bar{Z}$$

$$new\ cnf : (X \vee Y) \wedge (\bar{Y} \vee Z) \wedge (\bar{X} \vee \bar{Y} \vee \bar{Z})$$

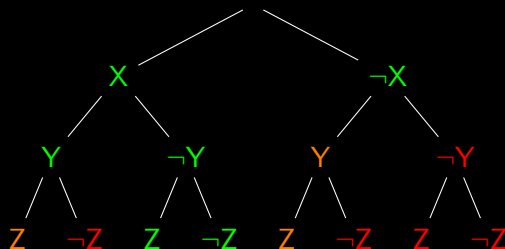
Model Enumeration using IPASIR-UP

IPASIR-UP extends IPASIR by an User Propagator

- ▶ `cb_check_found_model`
- ▶ `cb_has_external_clause`
- ▶ `cb_add_external_clause_lit`
- ▶ `notify_assignment`
- ▶ `notify_new_decision_level`
- ▶ `notify_backtrack`
- ▶ `cb_decide`
- ▶ `cb_propagate`
- ▶ `vb_add_reason_clause_lit`

Model Enumeration without Blocking Clauses

cnf: $(X \vee Y) \wedge (\bar{Y} \vee Z)$



Model Enumeration without Blocking Clauses

Continue the search after finding a model

```
bool cb_check_found_model(model &m):  
    ...  
    new_dl = highest_dl_with_positive_decision()  
    if (new_dl <= 0) return true  
    solver->force_backtrack(new_dl - 1)  
    ...  
    return false
```

Model Enumeration without Blocking Clauses

Continue the search after backtracking

1. Propagate the negation of the old decision
2. Decide the negation of the old decision

Model Enumeration without Blocking Clauses

Propagate the negation of the old decision

```
bool cb_propagate():  
    ...  
    return -decisions.back()  
  
bool cb_add_reason_clause_lit(propagated_lit):  
    return propagated_lit  
    return -decisions[dl]  
    ...  
    return -decisions[1]  
    return 0
```

Model Enumeration without Blocking Clauses

Decide the negation of the old decision

```
bool cb_decide():  
    ...  
    if decision_count_per_level[dl + 1] == 1:  
        return -decisions[dl + 1]  
    ...
```

In `cb_check_found_model`:

```
# new_dl = highest_dl_with_positive_decision()  
new_dl = highest_dl_with_decision_count_one()
```

Projected Model Enumeration

Projected cnf file for $(X \vee Y) \wedge (\bar{Y} \vee Z)$

```
c 1,2
p cnf 3 2
1 2 0
-2 3 0
```

```
for (int var : get_projected_vars(cnf_file)) {
    solver.add_observed_var(var);
}
```

Minimizing Models

```
bool cb_check_found_model(model &m) {  
    // new_model = m;  
    for (int lit : m) {  
        new_model.push_back(lit);  
        if (sat(cnf, new_model)) break;  
    }  
  
    all_models.push_back(new_model);  
    return false;  
};
```

Proof

solver.trace_proof(proof_file);

Proof file for $(X \vee Y) \wedge (\bar{Y} \vee Z)$

```
i 1 1 2 0
```

```
i 2 -2 3 0
```

```
q 0
```

```
i 3 -3 -2 -1 0
```

```
i 4 -3 -2 1 0
```

```
...
```

```
i 6 -3 2 -1 0
```

```
...
```

```
i 10 3 2 -1 0
```

```
...
```

```
s UNSATISFIABLE
```

```
...
```

Proof about my program

- ▶ no identical models
- ▶ if minimized: models differ
- ▶ if not minimized: models are total
- ▶ if projected: cnf with model assumption is SAT
- ▶ if not projected: models satisfy cnf

Proof no identical models

Loop over all models.

Add every model to a set, if it is not already in there.

Also check that no literal occurs twice.

Proof if minimized: models differ

Check that all models pairwise contain at least one literal that occurs negative in the other.

Proof if not minimized: models are total

Check that all models contain all (projected) literals.

Proof if projected: cnf with model assumption is SAT

For each model:

- Create a new solver instance.

- Assume all literals from one model.

- Check that it is still SAT.

Proof if not projected: models satisfy cnf

For each model:

Check that it satisfies the cnf.

Thats about it

Showcase?